

Attention and Transformers

Ali Gooya

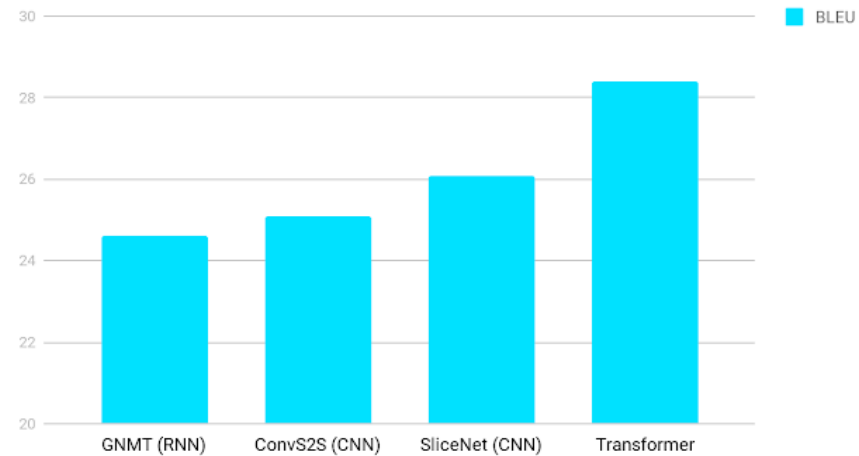
Learning outcomes

By the end of this lecture, you will:

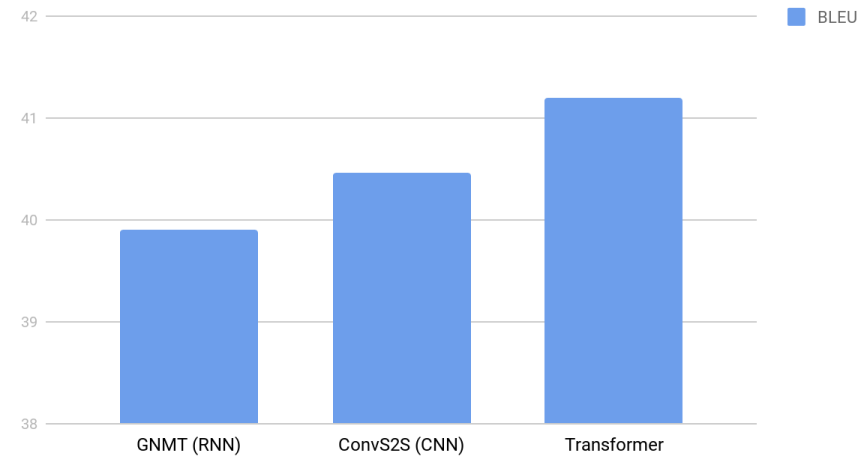
- Identify the limitations of RNNs as serial seq-to-seq converters
- Formulate the attention mechanism
- Differentiate the various components of the Transformer Networks

Improvement with transformer architecture

English German Translation quality

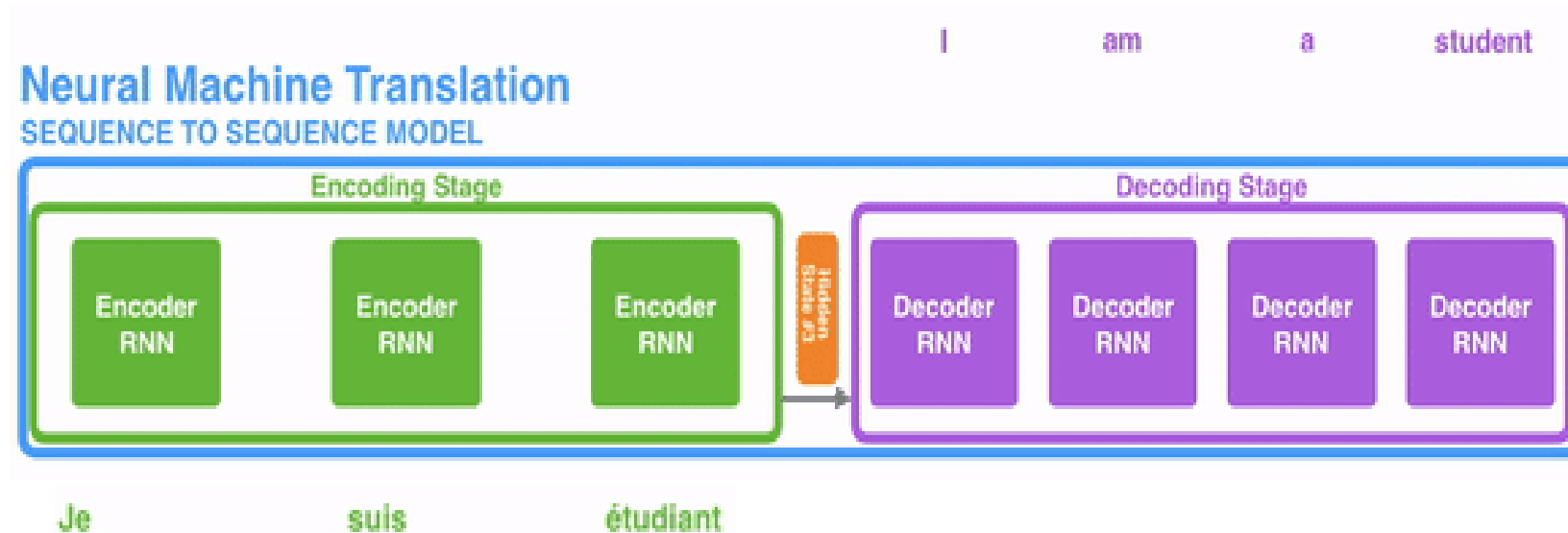


English French Translation Quality



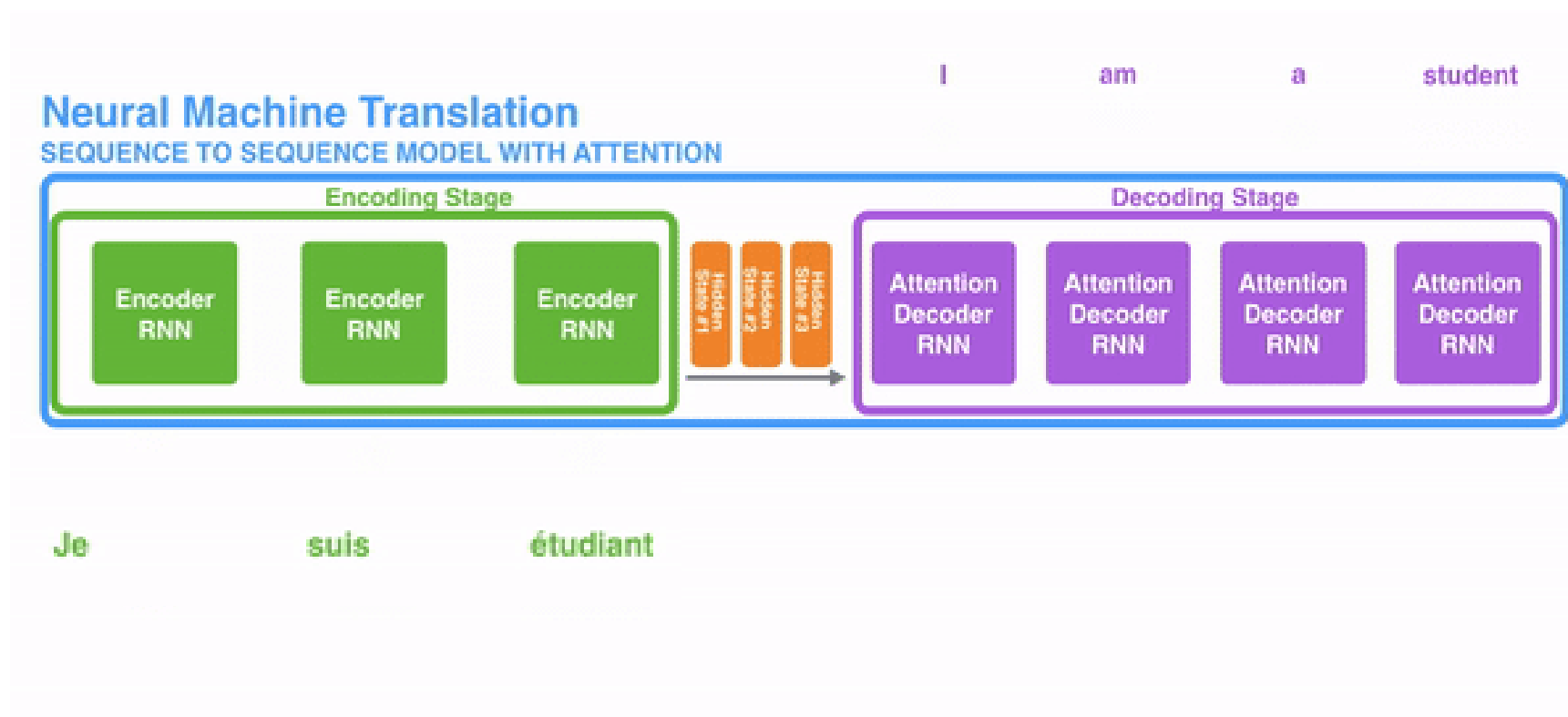
Why yet another DL model for sequences?

- RNNs 'shoe horn' the whole sequence history in a *context/state* variable.



- Sequential computations makes parallelization impossible.
- Vanishing gradients with increasing sequence length.
- No explicit modeling for long and short term dependencies.

- **‘Attention’** models explicit short-long range dependencies.



- The state variables from all input words are used by the decoder.
- Attention is computed as the similarity scores between state variables of encoder and decoder.

Attention at time step 4

1. Prepare inputs



h_1



h_2



h_3

Encoder
hidden
states



Decoder hidden
state at time step 4

2. Score each hidden state

13

9

9

scores

Attention weights for
decoder time step #4

3. Softmax the scores

0.96

0.02

0.02

softmax scores

4. Multiply each vector by
its softmaxed score



+



+



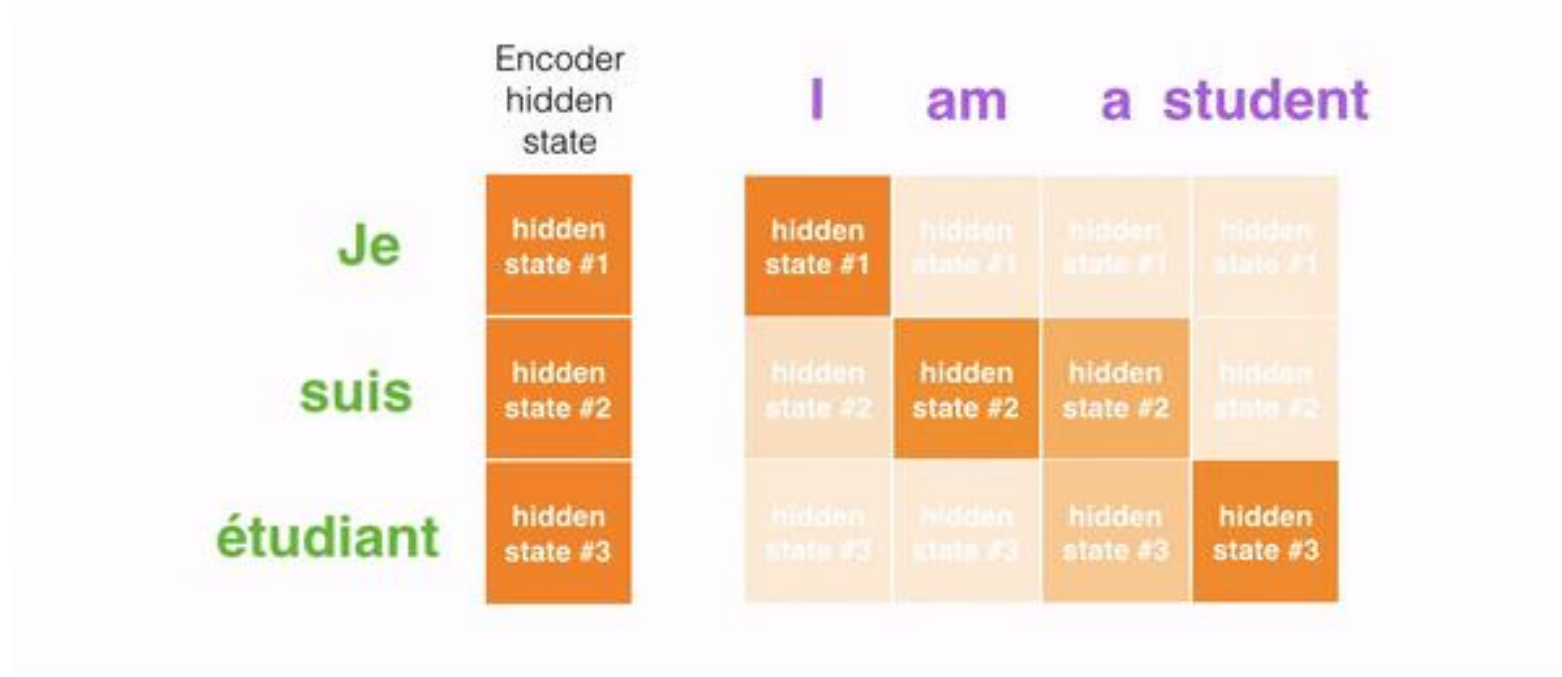
=

5. Sum up the weighted
vectors



Context vector for
decoder time step #4

- Attention weights obtained



- **Question:** Have we fully rectified the limitations of RNN?
- Not yet! translation is still sequential (slow training).
- This is solved by the Transformer Network (Vaswani et al, 2017).

Transformer Network (Attention is All You Need, Vaswani et al, 2017)

- Tokenises the words and uses their embeddings as input:

Example Phrase: "Tokenization is helpful."

A GPT tokenizer might see it like this:

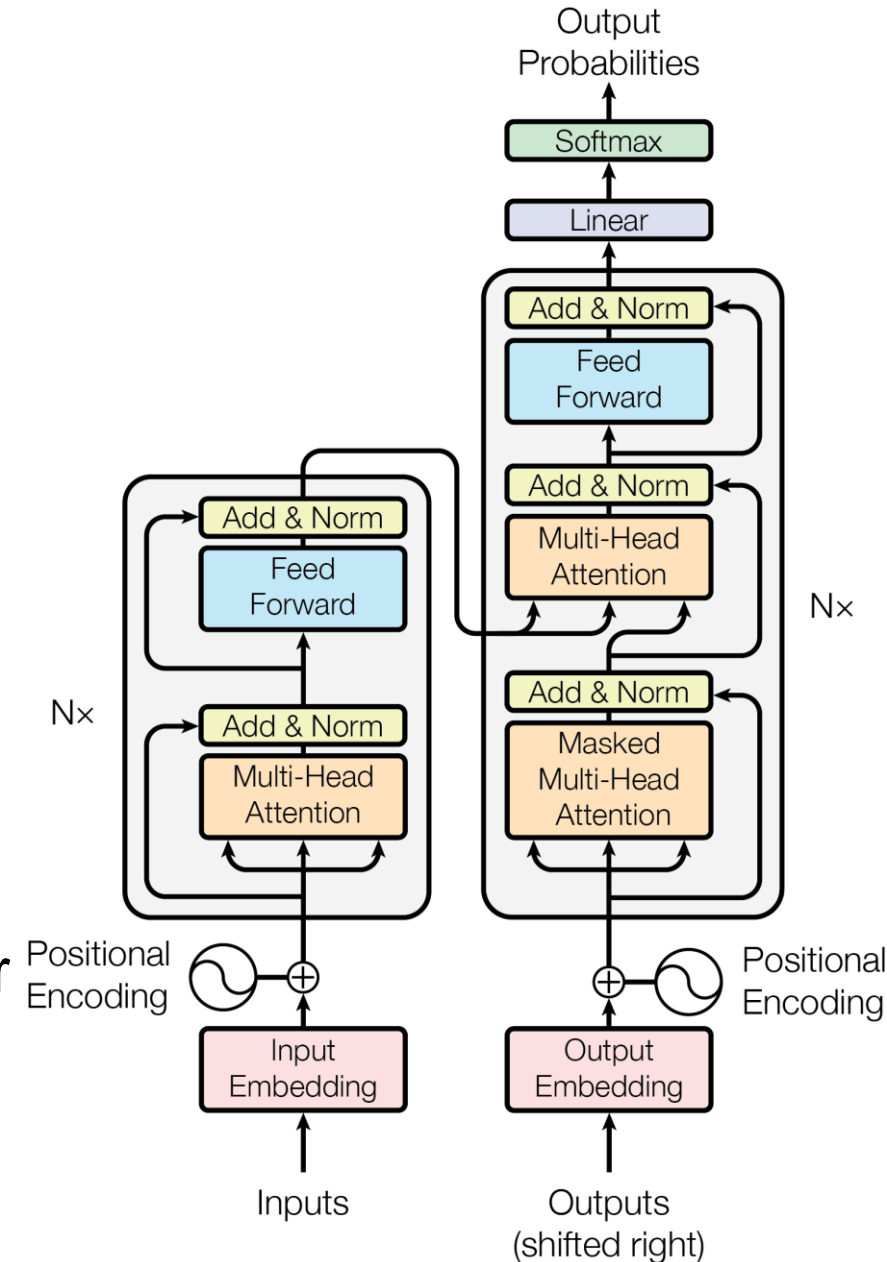
- Token
- ization
- (space)is
- (space)help
- ful
- .

Input sentences -> [34521, 674, 318, 1245, 402]

- Why do we do this?

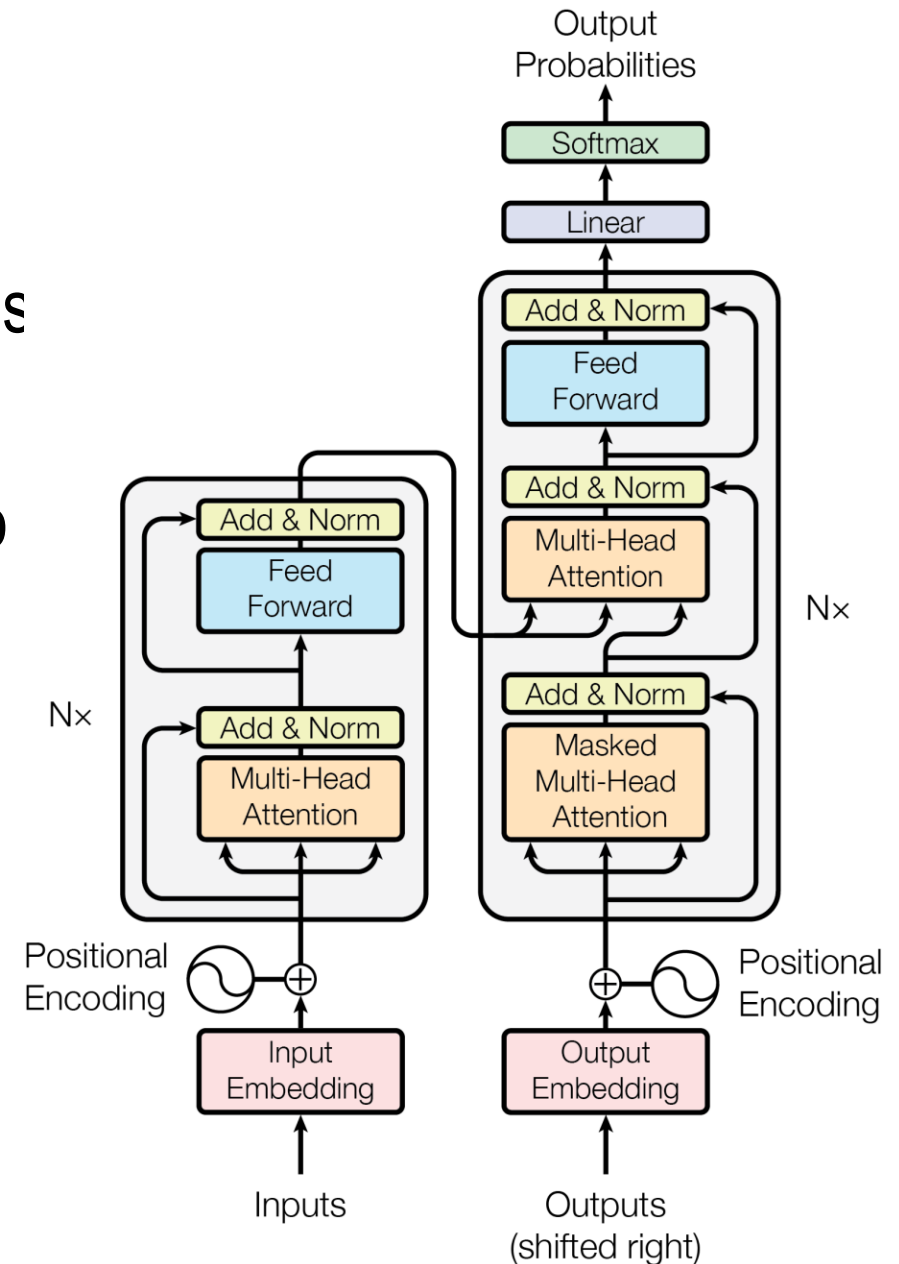
Efficiency: It recognizes "Token" and "ization" separately. If it later sees "Modernization," it already knows what "ization" means.

Vocabulary Size: It allows the model to understand millions of word combinations using a vocabulary of only 100,000 tokens.



Transformer Network (Attention is All You Need, Vaswani et al, 2017)

- Positional encodings establish the order of the tokens
- Uses a stack to transform the embeddings of tokens
- MHA units enable cross-talk between embeddings to identify the appropriate context
- Add & Norm normalises and adds them as residuals
- FF layer refines the embeddings privately
- The decoder is the base for chat Generative Pre-Trained (GPT) like models (gpt).



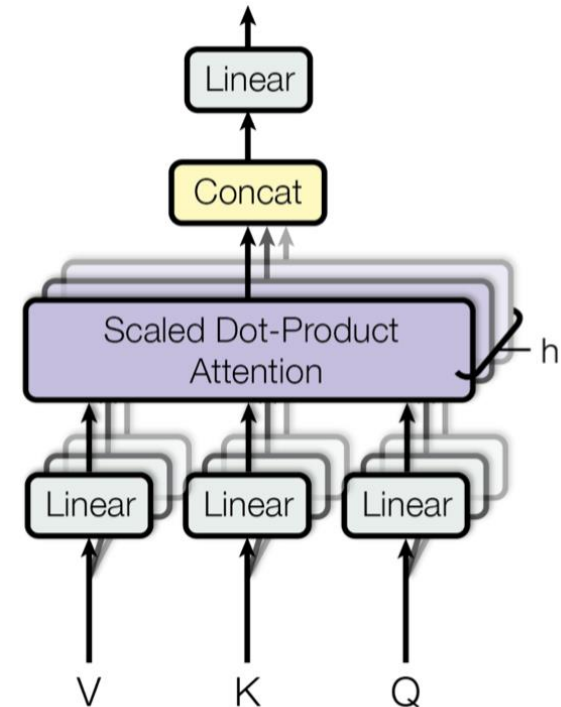
Attention in Transformer

- Attention: dot product of embeddings of queries (rows of Q) and keys (rows of K), normalised by dimension d and softmaxed.

- Transformation of embeddings:

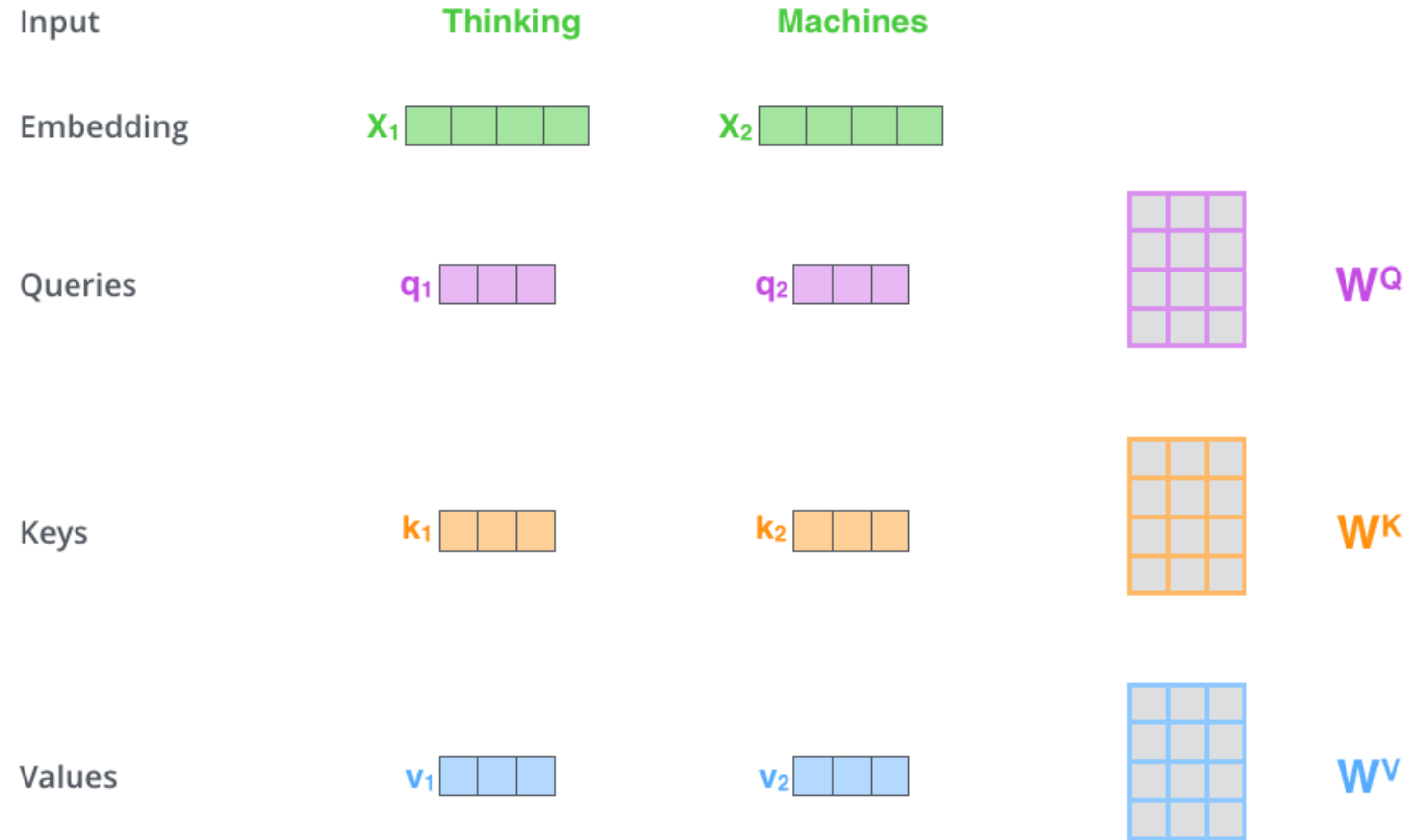
$$V \leftarrow \text{Attention} \left(\text{Softmax} \left(\frac{QK^T}{\sqrt{d}} \right) \right) \times \text{Values} \times V$$

- Can be computed in parallel (like CNN) via multi-head layers
- **Self-attention:** $Q = K$
- Decoder-encoder attention: Q from Decoder, K from encoder



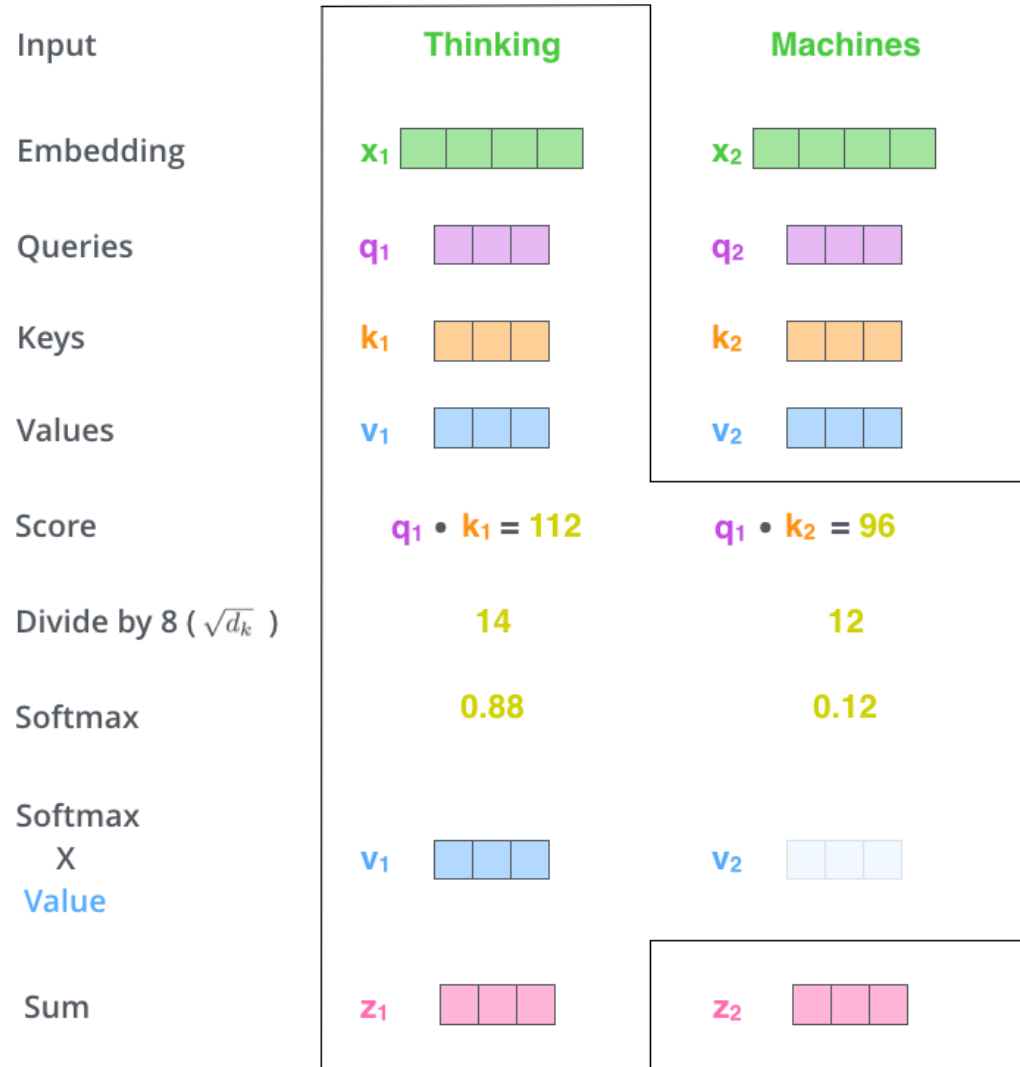
Self-attention

- Self attention is computed from **Queries**, **keys** and **values**.
- They are **abstractions**, calculated from the word embedding.
- The matrices W^Q , W^K , W^V are the weights trained during learning.



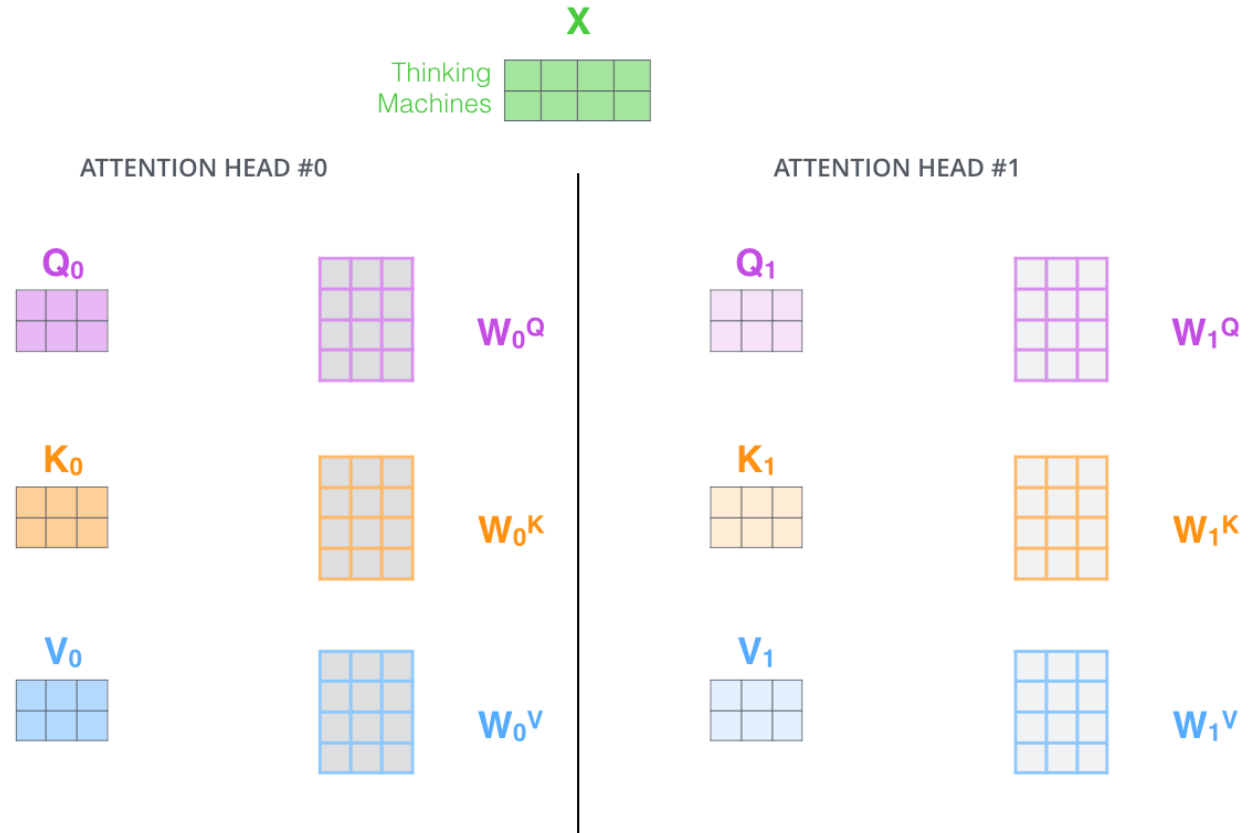
Self attention

- **Query:** Holds a representation for the current word
- **Keys:** Hold representation of all words in the sentence
- Their **product** form a score that indicate how the key word is related to the query.
- This means we create a new embedding blending words in the sentence that are related to the word.



Multi-head attention

- Instead of learning a single set of matrices, the transformer encoder learns multiple in parallel.
- The output of all heads is then concatenated and combined using a mixing matrix.



1) This is our input sentence*

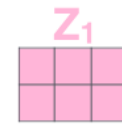
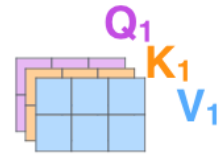
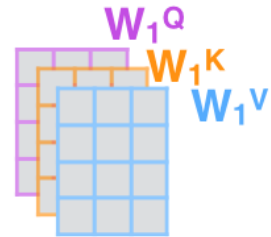
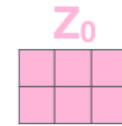
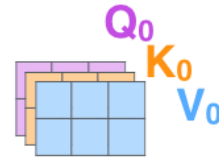
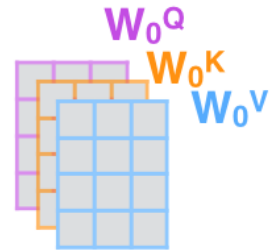
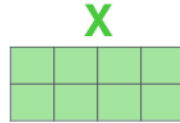
2) We embed each word*

3) Split into 8 heads. We multiply X or R with weight matrices

4) Calculate attention using the resulting $Q/K/V$ matrices

5) Concatenate the resulting Z matrices, then multiply with weight matrix W^O to produce the output of the layer

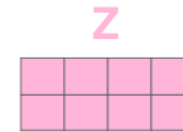
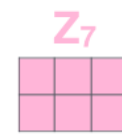
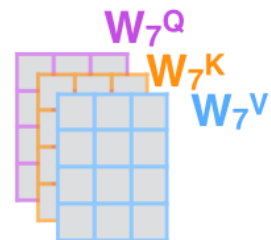
Thinking
Machines



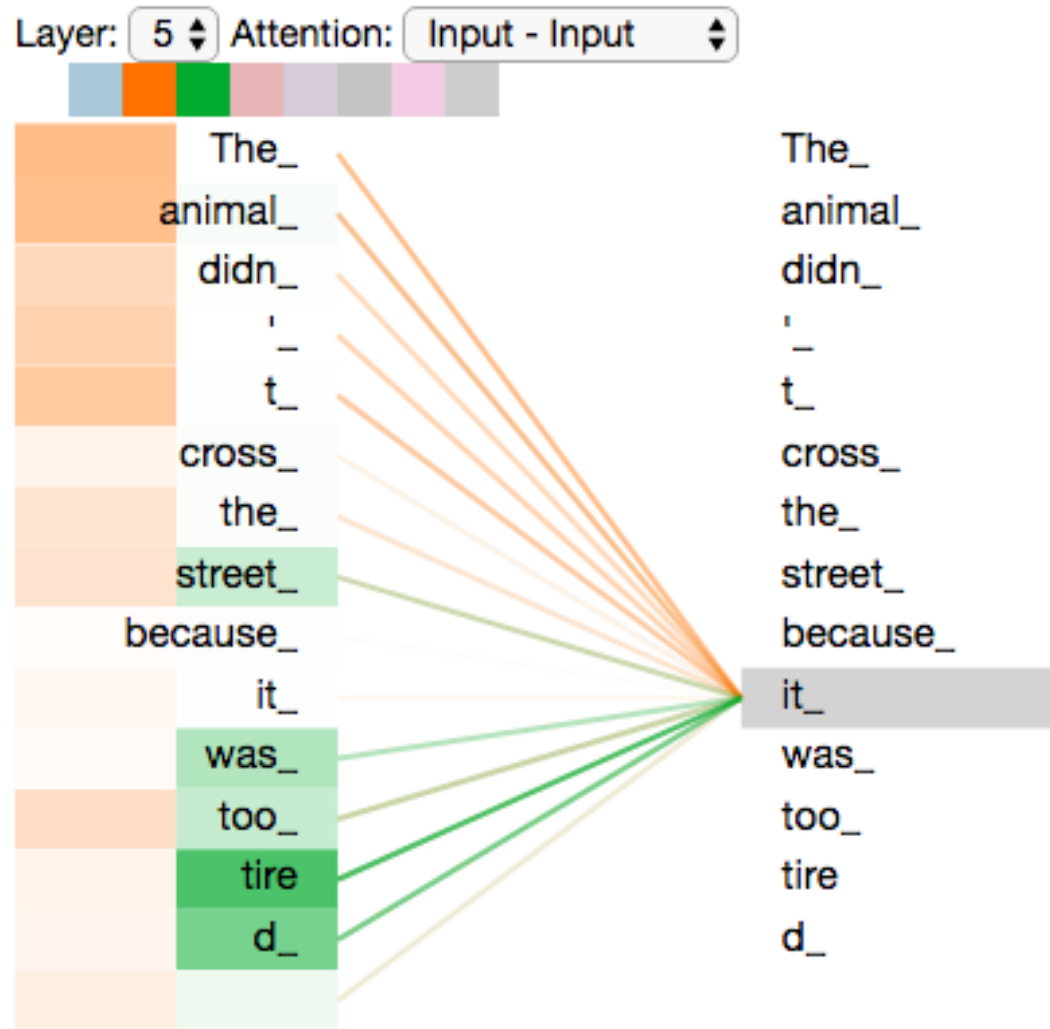
...

...

...



* In all encoders other than #0, we don't need embedding. We start directly with the output of the encoder right below this one



Example of Self-Attention

MHA mathematical recap

- Unmasked MHA recap:

$$Q_i = XW_i^Q, \quad K_i = XW_i^K, \quad V_i = XW_i^V$$

$$S_i = \frac{Q_i K_i^\top}{\sqrt{d_k}} \in \mathbb{R}^{n \times n}$$

$$(S_i)_{ab} = \frac{\langle q_{ia}, k_{ib} \rangle}{\sqrt{d_k}}$$

$$A_i = \text{softmax}(S_i)$$

$$\boxed{\text{MHA}(X) = [A_1 X W_1^V \mid \dots \mid A_h X W_h^V] W^O}$$

Masked MHA

- Masked MHA allows causal training – each token is blind to the next tokens

- In autoregressive training:

$$p(x_1, \dots, x_n) = \prod_{t=1}^n p(x_t \mid x_{<t})$$

- So token t **must not attend to future tokens** $t + 1, \dots, n$.

$$M = \begin{pmatrix} 0 & -\infty & -\infty & \dots \\ 0 & 0 & -\infty & \dots \\ 0 & 0 & 0 & \dots \\ \vdots & & & \ddots \end{pmatrix}$$

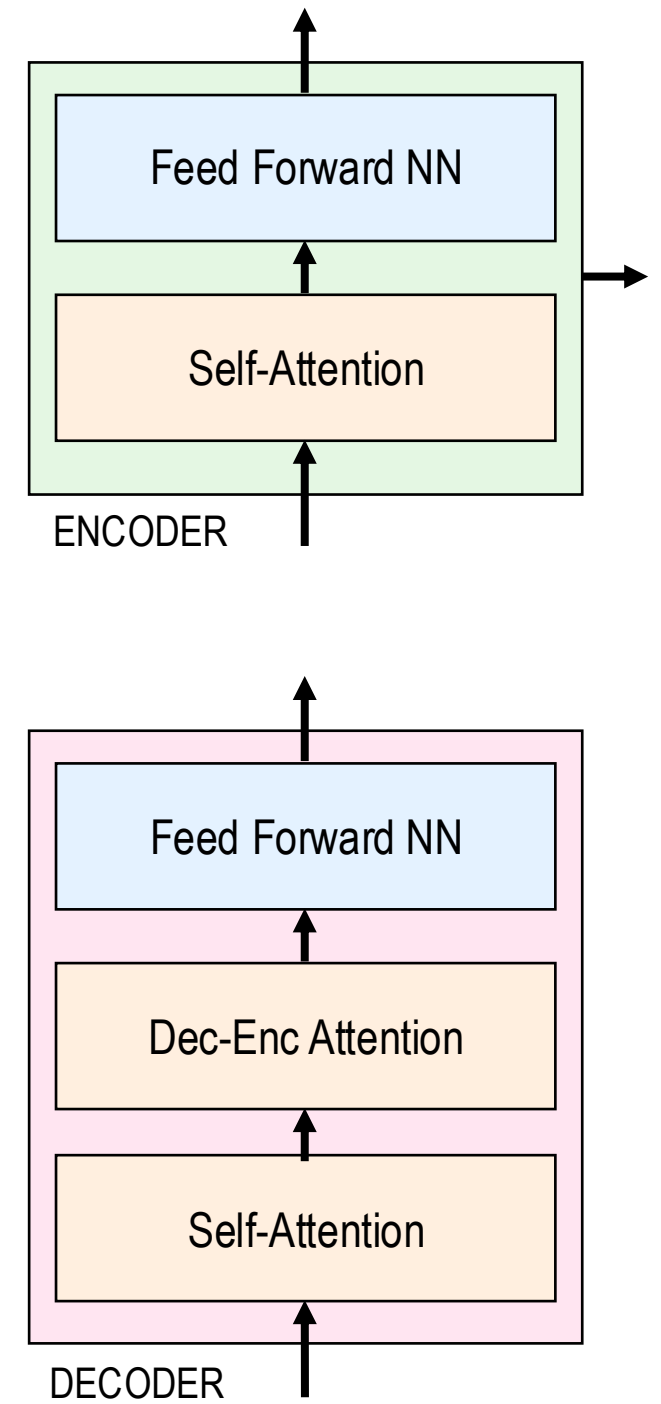
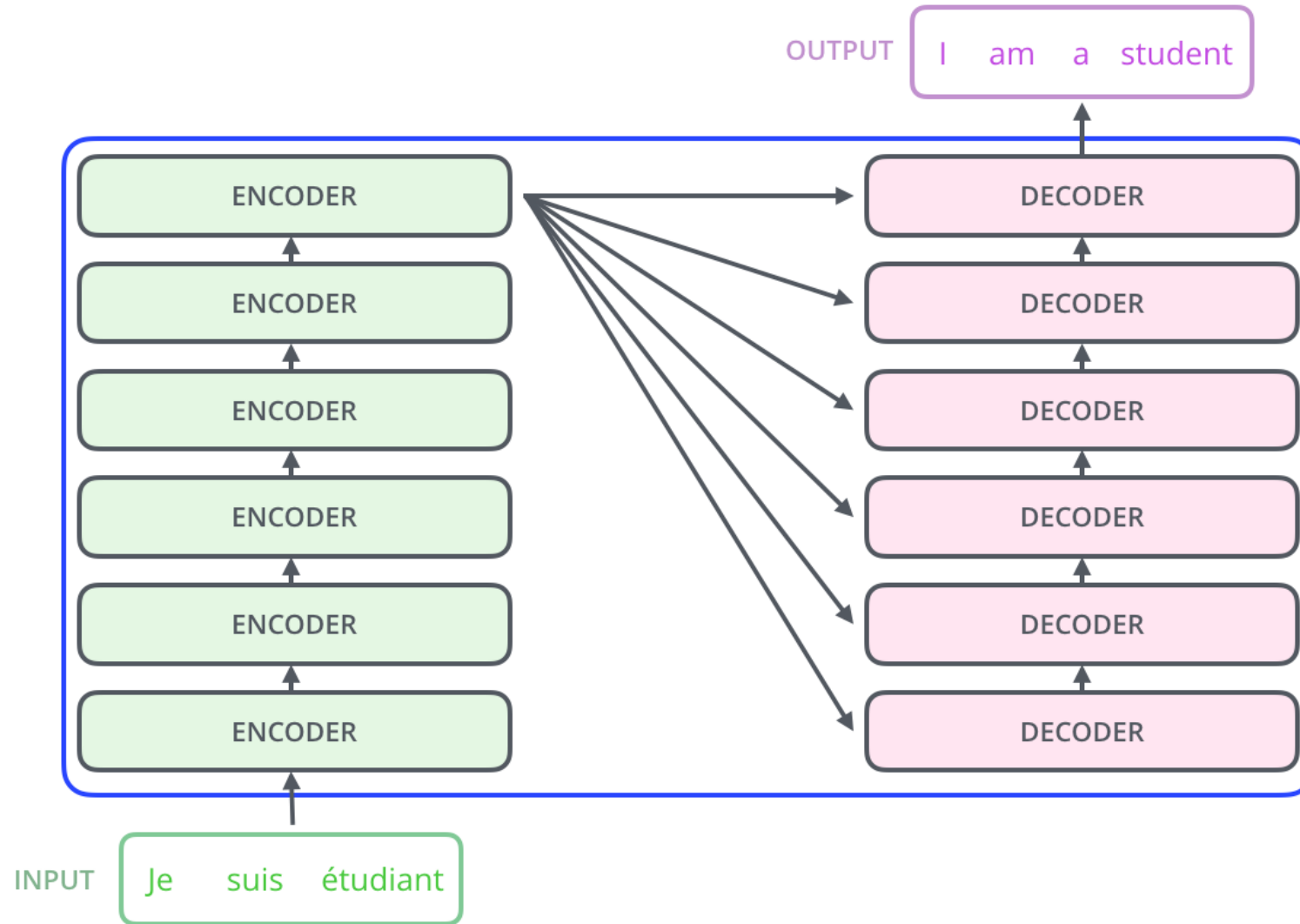
$$\tilde{S}_i = \frac{Q_i K_i^\top}{\sqrt{d_k}} + M$$

$$A_i = \text{softmax}(\tilde{S}_i)$$

$$\exp(-\infty) = 0$$

$$(A_i)_{ab} = 0 \quad \text{if } b > a$$

Transformer network



Conclusions:

- Attention captures long-short range of dependencies precisely.
- Transformer networks use parallelization and self-attention to address the limitations of the RNNs.
- Two types for attention:
 - Self-attention – used to transform word embeddings
 - Decoder-encoder attention – used to establish cross-modality correspondences.